

BUILD SPECIFICATION / PART 2 OF 2 // v1.0 // SEPTEMBER 2026

Low-Level Design

Food Delivery Platform - Enhancement

Design system, components, pages, schema diffs, API additions, backend fixes, testing and deployment

Contents

Contents	2
1. Agent Brief and Conventions	3
2. Repository Structure and Tooling	3
3. Design System - "Vivid Kitchen"	4
4. Component Library	7
5. Page Specifications	8
6. Data Model Changes	9
7. API: What Is Reused vs. What Is New	10
8. Backend Hardening Tasks (Exact Fixes)	11
9. Server Implementation Notes	11
10. Client Implementation Notes	13
11. Testing and QA	13
12. Build Phases and Acceptance Criteria	14
13. Deployment Runbook	15
14. Handover Requirements	16

1. Agent Brief and Conventions

This LLD assumes the existing backend described in section 6 (models) and section 7 (API) is correct - it reflects a real, hand-coded codebase, not a plan. Sections 15 and 16 list every backend change required; everything else in the backend is reused unchanged.

- Language: JavaScript. Backend stays CommonJS (matches the existing `"type": "commonjs"`). Frontend uses ES modules and JSX (Vite default).
- Naming: keep the existing backend naming (`thing.controller.js`, `thing.model.js`, `thing.routes.js`, `thing.validation.js`). New frontend files follow `PascalCase.jsx` for components, `useThing.js` for hooks.
- Formatting: Prettier (single quotes, semicolons, 100 columns). ESLint on both apps.
- Commits: conventional commits. Commit the existing backend unchanged first (`chore: import existing backend`), then one commit per fix or feature.
- Every list, grid and form in the new frontend implements loading, empty, error and success states.
- Do not touch `payment.controller.js` logic beyond what section 15 explicitly asks for - Cashfree integration is working and fragile to regress.

2. Repository Structure and Tooling

```

food-delivery/                                (git init here first; commit existing backend as-is)
  server/                                     existing backend, moved here unchanged, then hardened
    src/
      app.js server.js
      config/ bullmq.js cashfree.js redis.js cloudinary.js (new) logger.js (new)
      controllers/ (existing 9, unchanged in signature) upload.controller.js (new)
      middlewares/ auth.middleware.js error.middleware.js rateLimiter.middleware.js
                  validate.middleware.js upload.middleware.js (new, multer)
      models/      (existing 8, unchanged except image field shape)
      queues/      email.queue.js
      routes/      (existing 9) upload.routes.js (new)
      services/    (new: thin extraction of duplicated logic, incremental, not a rewrite)
      sockets/     index.js (new: Socket.IO auth + order/restaurant rooms)
      utils/       ApiError.js ApiResponse.js asyncHandler.js cache.js paginate.js
                  sendEmail.js logger.js (new, pino)
      validations/ (existing 9) upload.validation.js (new)
      workers/     email.worker.js (now started by `npm run worker`)
      cron/        index.js (now imported and started from server.js)
      scripts/     seed.js (new: demo restaurants, foods, users)
      tests/       auth, orders, payments, rbac (new)
      Dockerfile   (new)
      .env.example (new)
  client/                                     new React 18 + Vite SPA, all three roles
    public/        favicon.svg, placeholders/*.svg
    src/
      main.jsx App.jsx routes.jsx
      styles/index.css tokens, base layer
      config/site.js brand, nav per role
      content/      cuisineCategories.js, demoCopy.js
      lib/           api.js socket.js queryClient.js utils.js motion.js
      hooks/         useAuth useRestaurants useRestaurant useCart useCheckout
                    useOrders useOrder useOwnerOrders useSocketOrder useUpload
      context/       AuthContext.jsx CartContext.jsx
      components/ui/ Button Card Badge StatusPill Input Select Textarea
                    Modal Skeleton EmptyState Toast FilterBar PageHeader
                    Table Pagination ImageUploader
      components/layout/ Navbar Footer CustomerLayout OwnerLayout AdminLayout
                    ProtectedRoute RoleRoute
      components/customer/ RestaurantCard MenuItemCard CartDrawer AddressForm
                    OrderStatusTimeline OrderCard CategoryPills
      components/owner/ RestaurantProfileForm MenuItemForm OwnerOrderRow
      components/admin/ OwnerRequestRow UserRow RestaurantRow
      pages/customer/ Home Restaurants RestaurantDetail Cart Checkout
                    Orders OrderDetail Profile Addresses Login Register NotFound
      pages/owner/   Dashboard RestaurantProfile Menu Orders
      pages/admin/   Dashboard OwnerRequests Users Restaurants
      index.html vite.config.js tailwind.config.js postcss.config.js vercel.json

```

```
.github/workflows/ci.yml
docker-compose.yml (api + worker, for local parity with production)
package.json  README.md  .gitignore
```

2.1 New dependencies to add

App	New dependencies
server	multer, cloudinary, socket.io, pino, pino-http, rate-limit-redis, and (dev) jest or vitest, supertest, mongodb-memory-server
client	react, react-dom, react-router-dom, framer-motion, @tanstack/react-query, axios, socket.io-client, react-hook-form, @hookform/resolvers, zod, lucide-react, sonner, clsx, tailwind-merge, and (dev) vite, tailwindcss, eslint, prettier, vitest, @testing-library/react
server (remove)	razorpay (unused; uninstall)

2.2 Scripts to add

```
server: "start" node src/server.js (unchanged)
        "dev" node --watch src/server.js
        "worker" node src/workers/email.worker.js <- NEW, was never runnable before
        "seed" node src/scripts/seed.js <- NEW
        "test" jest (or vitest run) <- NEW
client: "dev" vite | "build" vite build | "preview" vite preview | "test" vitest run
root: "dev" concurrently server:dev, server:worker, client:dev
```

3. Design System - "Vivid Kitchen"

3.1 Colour tokens

Token	Value	Usage
--bg	#FFFBF7	Page background (warm off-white, not stark white)
--surface	#FFFFFF	Cards, inputs, modals
--surface-muted	#FFF3EA	Alternating sections, subtle highlight blocks
--border	#F0E4D8	Card and input borders, dividers
--text	#231A14	Headings and primary text
--text-2	#6B5A4E	Body copy, secondary text
--text-3	#A6968A	Captions, placeholders
--primary	#FF5A1F (hover #E8480F)	Primary buttons, active nav, prices
--secondary	#16A34A (hover #128A3E)	Success, "open now", confirm actions
--accent	#FF2D78	Sparingly: promo badges, featured tags, wishlist
--warning / --danger	#F59E0B / #EF4444	Pending / cancelled states, form errors
--gradient-warm	linear-gradient(135deg, #FF5A1F 0%, #FF2D78 100%)	Hero blobs, featured badge, primary button on hover
Status colours	PLACED #F59E0B, CONFIRMED #3B82F6, PREPARING #8B5CF6, OUT_FOR_DELIVERY #FF5A1F, DELIVERED #16A34A, CANCELLED #EF4444	StatusPill background tint + matching text/icon colour, identical everywhere

3.2 Typography

Fonts: **Poppins** 600/700 for display and headings (rounded, friendly geometric sans - common in food apps), **Inter** 400/500/600 for body and UI, **JetBrains Mono** 500 only for order/reference IDs. Preconnect + swap, subset latin.

Style	Size / line-height	Weight	Notes
display	clamp(2.25rem, 5vw, 3.75rem) / 1.1	700	Home hero headline, Poppins
h2	clamp(1.75rem, 3.5vw, 2.5rem) / 1.2	600	Section titles
h3	1.25rem / 1.3	600	Card titles, restaurant names
price	1.125rem / 1	700	Tabular numerals, --primary colour
body	1rem / 1.65	400	Default
small	0.8125rem / 1.4	500	Meta, status pills, table cells

3.3 Layout, spacing and shape

Rule	Value
Container	max-width 1180px, horizontal padding 20-24px
Grid	Restaurant/menu cards: 1 / 2 / 3 / 4 columns at base / sm / md / lg, 20px gap
Radius	sm 10px (inputs, pills half-height) md 16px (buttons, small cards) lg 20px (cards) xl 28px (hero panels, modals) full (category pills, status pills, avatar)
Card shadow	0 1px 2px rgba(35,26,20,.04), 0 12px 24px -12px rgba(35,26,20,.15); hover: 0 20px 32px -12px rgba(255,90,31,.20)
Breakpoints	sm 640 md 768 lg 1024 xl 1280
Icons	lucide-react, 20px inline / 24px feature, stroke width 2
Focus ring	2px solid --primary, 2px offset, on every interactive element

3.4 Motion

Token	Value
Durations	micro 150ms base 250ms reveal 450ms
Easing	cubic-bezier(0.22, 1, 0.36, 1)
Card hover	translateY(-3px) + image scale(1.05, 400ms) + shadow grows
Add to cart	button label crossfades to a checkmark for 900ms, cart icon in navbar does a 200ms scale bump, item count increments with a spring
Status timeline	progress line width animates on status change; the current-step dot pulses (opacity 0.6 to 1, 1.4s loop) until DELIVERED
Filters	active pill background slides via <code>layoutId</code> , 250ms
Reduced motion	disables card lift/zoom, pulse and pill-slide animations; opacity-only fallback

3.5 Effect recipes

- **Hero blob:** two large (500-650px) blurred circles behind the hero, warm gradient at 18-24% opacity, `blur-3xl`, subtle 20s float (transform only). Only on Home.

- **Floating cart bar:** a fixed bottom-right (desktop) / bottom (mobile) pill showing item count and subtotal, `backdrop-blur-md` on a semi-opaque white background, slides up when the cart has items, tapping opens `CartDrawer`.
- **StatusPill:** rounded-full, coloured background at 12% opacity of the status colour, coloured text and a small dot or icon in the full status colour - one component, six colour variants, used identically in customer, owner and admin views.
- **Category pills:** horizontally scrollable row, active pill filled with `--gradient-warm`, inactive pills outlined; `layoutId="category-pill"` for the sliding highlight.
- **Order timeline:** horizontal on desktop, vertical on mobile; five nodes (PLACED to DELIVERED), a connecting line that fills with `--secondary` up to the current step, current node pulses, cancelled orders show a red terminal node instead of the remaining steps.
- **Image upload preview:** dashed-border drop zone that becomes a live thumbnail with a hover overlay (change / remove) once an image is chosen or already set.

3.6 Tailwind configuration (excerpt)

```
// client/tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.js,jsx'],
  theme: {
    container: { center: true, padding: '1.25rem', screens: { xl: '1180px' } },
    extend: {
      colors: {
        bg: 'var(--bg)', surface: 'var(--surface)', 'surface-muted': 'var(--surface-muted)',
        border: 'var(--border)', fg: 'var(--text)', 'fg-2': 'var(--text-2)', 'fg-3': 'var(--text-3)',
        primary: 'var(--primary)', secondary: 'var(--secondary)', accent: 'var(--accent)',
      },
      fontFamily: {
        display: ['Poppins', 'ui-sans-serif', 'sans-serif'],
        sans: ['Inter', 'ui-sans-serif', 'sans-serif'],
        mono: ['JetBrains Mono', 'ui-monospace', 'monospace'],
      },
      borderRadius: { sm: '10px', md: '16px', lg: '20px', xl: '28px' },
      keyframes: {
        float: { '0%': { transform: 'translate3d(0,0,0)' }, '100%': { transform: 'translate3d(30px,-24px,0)' } },
        pulseDot: { '0%,100%': { opacity: .6 }, '50%': { opacity: 1 } },
      },
      animation: { float: 'float 20s ease-in-out infinite alternate', pulseDot: 'pulseDot 1.4s ease-in-out infinite' },
    },
  },
};
```

3.7 Consistency rules (enforced)

- Every status shown anywhere in the app - order status, owner-request status, payment status - renders through the single `StatusPill` component with a fixed colour map, never a bespoke coloured `<span>`.
- Every list/table page across all three roles uses the same `PageHeader` + `FilterBar` + `Table/grid` + `Pagination` composition.
- No hard-coded colours, radii or shadows in component code - tokens and Tailwind theme names only.
- Restaurant and food images always render at a fixed aspect ratio (`aspect-[4/3]` cards, `aspect-square` thumbnails) with `object-fit: cover`, so grids never reflow while loading.
- One `Button` component (primary/secondary/ghost x sm/md/lg) used everywhere, including inside owner and admin dashboards.

4. Component Library

4.1 UI primitives (components/ui)

Component	Props	Behaviour
Button	variant, size, loading, iconRight, to/onClick	Pill or rounded-md per size; spinner on loading; disabled state dims and blocks pointer events
Card	interactive, className, children	Base surface card; <code>interactive</code> adds hover lift + shadow growth
StatusPill	status (order/payment/ownerRequest enum)	Maps to colour + icon + label from one shared status map (section 3.7)
PageHeader	title, subtitle, actions (node)	Consistent page-top layout used by every list/detail page in every role
FilterBar	searchProps, filters[] (select/chip groups), onReset	Wraps search input + filter controls; collapses to a "Filters" sheet on mobile
Table	columns, rows, loading, empty	Used by owner/admin directories; skeleton rows while loading, <code>EmptyState</code> when empty
Pagination	page, totalPages, onChange	Numbered + prev/next, disabled at bounds
Modal	open, onClose, title, children	Focus trap, escape to close, used for delete confirmations and image lightbox
ImageUploader	value, onChange, aspect	Drop zone / preview / change / remove; shows upload progress; calls <code>useUpload</code>
Skeleton / EmptyState / Toast	—	Shape-matched skeletons per card type; friendly empty illustrations with a call to action; sonner-based toasts for success/error

4.2 Layout components

Component	Specification
Navbar (customer)	Logo, location chip (static text v1), search shortcut, cart icon with live count, account menu (orders, addresses, profile, logout, and "become an owner" if role is User)
OwnerLayout / AdminLayout	Left sidebar with role-specific nav (Dashboard, Restaurant, Menu, Orders for owner; Dashboard, Owner requests, Users, Restaurants for admin), top bar with page title and logout
ProtectedRoute / RoleRoute	Redirects to /login if unauthenticated (via <code>GET /auth/me...</code> i.e. the existing <code>/api/users/me</code>); RoleRoute additionally checks <code>user.role</code> and redirects to the correct home for that role
CartDrawer	Slide-over panel: line items with quantity steppers, subtotal, "Go to checkout" button; warns and offers to clear the cart if adding an item from a different restaurant (matches the backend's single-restaurant rule)

4.3 Feature components

Component	Specification
RestaurantCard	Cover image (4:3), open/closed StatusPill-style badge, name, short address, category chips; links to <code>/restaurants/:id</code>
MenuItemCard	Square thumbnail, name, description (2-line clamp), price, availability, quantity stepper once in cart, "Add" button with the add-to-cart micro-interaction
OrderStatusTimeline	Renders the five-step order status (section 3.5); subscribes to <code>useSocketOrder(orderId)</code> for live updates and falls back to polling <code>GET /api/orders/:id</code> every 10s if the socket is disconnected
OrderCard	Compact order summary for the history list: restaurant name, item count, total, StatusPill, placed date, link to detail
RestaurantProfileForm (owner)	Name, description, address, cover image (ImageUploader), open/closed toggle - maps directly to the existing restaurant fields
MenuItemForm (owner)	Name, description, price, category, image (ImageUploader), availability toggle - maps directly to the existing food fields
OwnerOrderRow	Order summary + a single "Advance to <next status>" button driven by the existing status enum order, plus a StatusPill; live-updates via the owner's restaurant Socket.IO room
OwnerRequestRow (admin)	Requester name/email, reason, StatusPill, Approve/Reject buttons with a reason field on reject

5. Page Specifications

5.1 Customer app

Page	Content and behaviour
Home (/)	Hero with gradient blobs, headline, search bar; horizontally scrollable cuisine <code>CategoryPills</code> ; "Featured restaurants" and "Open near you" grids (reuses <code>GET /api/restaurants</code> with different query params); empty/loading states via skeleton <code>RestaurantCards</code>
Restaurants (/restaurants)	FilterBar (search, category, open-only toggle) bound to <code>GET /api/restaurants</code> query params; responsive grid; pagination; empty state with "clear filters" action
Restaurant detail (/restaurants/:id)	Cover image, name, address, open/closed pill; menu grouped by <code>category</code> (client-side grouping of <code>GET /api/foods?restaurantId=</code>), in-page category jump nav, search box filtering the loaded menu; floating cart bar appears once an item is added
Cart (/cart)	Line items with steppers (calls the existing <code>/api/cart</code> update/remove endpoints), price breakdown mirrors <code>GET /api/checkout</code> (subtotal, delivery fee, tax, total), "Proceed to checkout" disabled if empty
Checkout (/checkout)	Address picker (existing <code>/api/addresses</code> , "add new" inline), full price summary from <code>GET /api/checkout</code> , "Pay now" calls <code>POST /api/payments/create</code> then redirects into the Cashfree session; on return, polls/confirm via the existing verify/callback flow
Order tracking (/orders/:id)	<code>OrderStatusTimeline</code> , item list, delivery address, cancel button shown only while status is PLACED or CONFIRMED (matches backend rule), "Contact restaurant" mailto/phone if available
Order history (/orders)	Status filter chips, paginated <code>OrderCard</code> list, empty state with "Browse restaurants" CTA
Profile / Addresses (/profile, /addresses)	Profile edit and password change (existing endpoints); address book CRUD; a card showing "become a restaurant owner" - submits <code>POST /api/owner-requests</code> and shows the current request's StatusPill if one exists
Login / Register	Existing auth endpoints; friendly inline validation; register offers no role selector (always defaults to User, matching backend default)

5.2 Restaurant owner dashboard

Page	Content and behaviour
Dashboard (/owner)	If the owner has no restaurant yet, a prominent "Create your restaurant" empty state; otherwise open/closed toggle, today's order count, and a shortcut to Orders
Restaurant profile (/owner/restaurant)	RestaurantProfileForm ; creating a restaurant here is what backs the "no restaurant yet" empty state on Dashboard
Menu manager (/owner/menu)	Table/grid of the owner's food items with availability toggle inline; "Add item" opens MenuItemForm in a modal; delete with confirmation
Orders (/owner/orders)	Live queue via Socket.IO (new order events join the restaurant's room); filter by status; each row is OwnerOrderRow with a one-tap "advance status" action calling the existing PATCH /api/orders/:id/status

5.3 Admin dashboard

Page	Content and behaviour
Dashboard (/admin)	Counts: total users, total restaurants, pending owner requests (from existing list endpoints, counted client-side or via a small new GET /api/admin/stats if the agent judges it cleaner than aggregating client-side - optional, not required)
Owner requests (/admin/owner-requests)	Table of requests with status filter; Approve / Reject actions call the existing endpoints; reject requires a short reason
Users (/admin/users)	Read-only searchable, paginated table of GET /api/users (already admin-only): name, email, role, joined date
Restaurants (/admin/restaurants)	Read-only searchable, paginated table of GET /api/restaurants : name, owner, open/closed, item count

5.4 Global states

- **404:** friendly branded not-found page with role-aware "go home" link (customer home, owner dashboard, or admin dashboard depending on the logged-in role).
- **Socket disconnect banner:** a small non-blocking banner on the order-tracking and owner-orders pages if the socket drops, explaining that updates may be delayed and offering a manual refresh.
- **Payment failure:** checkout shows a clear retry action if Cashfree verification fails, without losing the cart.

6. Data Model Changes

All 8 existing schemas (User, Restaurant, Food, Cart, Order, Address, Payment, OwnerRequest) are correct as documented in the status report and are reused with **one change**: the plain `image: String` field becomes a small embedded object so an image can be replaced or removed cleanly.

```
// BEFORE (existing, both Restaurant and Food models)
image: { type: String }

// AFTER
image: {
  url: { type: String, default: '' },
  publicId: { type: String, default: '' }, // Cloudinary public id, used to delete on replace
},
```

This is an additive, backward-compatible change: existing documents with a plain string simply have `image.url` populated by a one-line migration script (`node src/scripts/migrate-image-field.js`) run once before the new upload feature ships. No other model changes are required for v1.

7. API: What Is Reused vs. What Is New

7.1 Reused unchanged (already correct per the status report)

All endpoints under `/api/users`, `/api/restaurants`, `/api/foods`, `/api/cart`, `/api/checkout`, `/api/orders`, `/api/addresses`, `/api/owner-requests`, and `/api/payments` are used by the new frontend exactly as they exist today. Do not change their request/response shapes.

7.2 New endpoints

Method + path	Auth	Purpose
POST <code>/api/uploads/image</code>	Authenticated (restaurantOwner for restaurant/food images)	multipart field <code>image</code> ; validates MIME (jpeg/png/webp) and size (max 4 MB); uploads to Cloudinary folder <code>food-delivery/{restaurants foods}</code> ; returns <code>{ url, publicId }</code>
DELETE <code>/api/uploads/image/:publicId</code>	Authenticated, ownership-checked	Deletes an image from Cloudinary (URL-encoded <code>publicId</code>); called when a food/restaurant image is replaced or removed
GET <code>/api/admin/stats</code>	admin (optional, see 5.3)	Returns <code>{ users, restaurants, pendingOwnerRequests }</code> counts, only if the agent chooses this over client-side aggregation

7.3 Real-time events (Socket.IO, new)

Event	Direction	Payload	Notes
connection (auth)	client to server	JWT read from the existing <code>accessToken</code> cookie at handshake	Reject the connection if the token is missing or invalid
join:order	client to server	<code>{ orderId }</code>	Server verifies the connected user owns the order before joining the room <code>order:{id}</code>
join:restaurant	client to server	<code>{ restaurantId }</code>	Server verifies the connected user owns the restaurant before joining room <code>restaurant:{id}</code>
order:status	server to client	<code>{ orderId, status, updatedAt }</code>	Emitted to <code>order:{id}</code> and <code>restaurant:{restaurantId}</code> whenever <code>PATCH /api/orders/:id/status</code> or the cancel endpoint succeeds
order:new	server to client	the created order (populated)	Emitted to <code>restaurant:{restaurantId}</code> when <code>POST /api/orders</code> succeeds, so the owner queue updates live

Socket.IO attaches to the same HTTP server Express already uses (`http.createServer(app)` then `new Server(httpServer, { cors: { ... } })`), so no new port or process is needed for it - only the worker (BullMQ) runs as a second process.

8. Backend Hardening Tasks (Exact Fixes)

Every row below ties to a specific finding in the status report. Apply each one, write or update a test that proves it, then move to the next.

#	Issue (from status report)	Fix
1	Order placement and cart clearing are two separate uncommitted writes	Wrap <code>Order.create()</code> and <code>cart.save()</code> (clearing items) in a single <code>mongoose.startSession()</code> transaction in <code>order.controller.js</code> ; roll back both on any failure
2	Cookies set with hardcoded <code>secure: false</code>	In <code>user.controller.js</code> , set <code>secure: process.env.NODE_ENV === "production"</code> and an explicit <code>sameSite (lax</code> if same-site deployment, <code>none</code> + <code>secure: true</code> if the API and client end up on different domains)
3	BullMQ email worker and cron are never started	Add <code>"worker": "node src/workers/email.worker.js"</code> to <code>package.json</code> and run it as a separate Render service; import and start <code>src/cron/index.js</code> from <code>server.js</code> (or run it inside the worker process, not the API process, to avoid duplicate schedules if the API scales to multiple instances)
4	No image upload pipeline; <code>image</code> is a plain URL string	Add Multer memory storage + the Cloudinary SDK; new <code>upload.controller.js/upload.routes.js</code> (section 7.2); update the Restaurant/Food models per section 6
5	In-memory rate limiter store does not scale across instances	Swap the store in <code>ratelimiter.middleware.js</code> to <code>rate-limit-redis</code> , reusing the existing Redis connection
6	Unused <code>razorpay</code> dependency	Run <code>npm uninstall razorpay</code> in <code>server/</code>
7	<code>package.json</code> "main" points to a non-existent <code>index.js</code>	Set <code>"main": "src/server.js"</code>
8	Fat controllers, empty <code>services/</code> folder	Incrementally extract clearly duplicated logic (for example the ownership check repeated across restaurant/food controllers, and the price/tax/delivery-fee calculation shared between checkout and order) into <code>services/</code> . Do not do a full rewrite - only move logic that is genuinely duplicated or heavily tested
9	Unformatted <code>console.log/console.error</code> everywhere	Introduce <code>pino (utils/logger.js)</code> and <code>pino-http</code> middleware; replace direct console calls with <code>logger.info/error</code> , including a request ID per request
10	No automated tests	Add the test suite in section 12
11	No Docker, no CI, no Git repo	Section 15 and 16

9. Server Implementation Notes

9.1 server.js startup (after hardening)

```
const httpServer = http.createServer(app);
const io = new Server(httpServer, { cors: { origin: env.CLIENT_ORIGINS, credentials: true } });
require('./sockets')(io); // auth handshake + room joins (section 7.3)
app.set('io', io); // controllers emit via req.app.get('io')

connectDB()
  .then(() => httpServer.listen(env.PORT, () => logger.info(`API listening on ${env.PORT}`)))
  .catch((err) => { logger.error(err, 'DB connection failed'); process.exit(1); });
// NOTE: cron and the email worker run in the separate worker process, not here.
```

9.2 Emitting order events from existing controllers

```
// inside order.controller.js, after a successful status update or cancellation:
const io = req.app.get('io');
io.to(`order:${order._id}`).emit('order:status', { orderId: order._id, status: order.status, updatedAt: order.updatedAt });
io.to(`restaurant:${order.restaurant}`).emit('order:status', { orderId: order._id, status: order.status });
// and after order creation:
io.to(`restaurant:${order.restaurant}`).emit('order:new', populatedOrder);
```

9.3 Upload controller (new)

```
// upload.middleware.js - multer memory storage, 4MB limit, jpeg/png/webp only
// upload.controller.js
exports.uploadImage = asyncHandler(async (req, res) => {
  if (!req.file) throw new ApiError(400, 'No image provided');
  const folder = req.query.type === 'food' ? 'food-delivery/foods' : 'food-delivery/restaurants';
  const result = await uploadBufferToCloudinary(req.file.buffer, folder); // streamifier + cloudinary.uploader.upload_stream
  res.status(201).json(new ApiResponse(201, { url: result.secure_url, publicId: result.public_id }));
});
```

9.4 Seed data (server/src/scripts/seed.js, new)

Seeds	Detail
3 demo users	One customer (User), one restaurantOwner, one admin - printed credentials at the end of the script, never committed to the repo
4 restaurants	Distinct cuisines (e.g. Italian, Indian, Japanese, Burgers), each owned by the seeded restaurantOwner, each with a placeholder cover image
6-8 foods per restaurant	Realistic names, prices, categories (starters/mains/desserts/drinks), placeholder square images
Idempotent	Upserts by email/name; safe to re-run; refuses to run when <code>NODE_ENV=production</code> unless <code>--force</code> is passed

9.5 Environment variables (additions)

Variable	Purpose
CLOUDINARY_CLOUD_NAME / CLOUDINARY_API_KEY / CLOUDINARY_API_SECRET	New - image uploads
CLIENT_ORIGINS	New - CORS and Socket.IO allowlist (comma-separated), replaces the single hardcoded <code>localhost:3000</code> origin
NODE_ENV	Already exists - now actually read for cookie <code>secure</code> and rate-limit behaviour
LOG_LEVEL	New - pino level (info in production, debug in development)

10. Client Implementation Notes

10.1 Routing and role guards

Path group	Guard	Loading
/, /restaurants, /restaurants/:id, /login, /register	Public	Eager for Home, lazy for the rest
/cart, /checkout, /orders*, /profile, /addresses	ProtectedRoute (any authenticated role)	Lazy, customer chunk
/owner/*	RoleRoute(["restaurantOwner"])	Lazy, separate owner chunk
/admin/*	RoleRoute(["admin"])	Lazy, separate admin chunk

10.2 Data and real-time layer

- `lib/api.js`: Axios, `withCredentials: true`, `baseUrl = VITE_API_BASE_URL || "/api"`, normalises errors from the existing `{ success, error }` envelope.
- `lib/socket.js`: single Socket.IO client instance created once the user is authenticated; disconnected on logout.
- `useSocketOrder(orderId)`: joins `order:{id}`, listens for `order:status`, updates the TanStack Query cache for that order directly (no extra fetch); on socket disconnect for more than 8s, falls back to a 10s poll of `GET /api/orders/:id`.
- `useOwnerOrders(restaurantId)`: joins `restaurant:{id}`, listens for both `order:new` and `order:status`, prepends/updates the owner's order list in the query cache.
- `useUpload()`: wraps `POST /api/uploads/image` with progress via Axios `onUploadProgress`, used by `ImageUploader`.

10.3 Performance and accessibility checklist

- Role-based code splitting: a customer session never downloads owner or admin JS.
- Images: explicit width/height or `aspect-*` classes, `loading="lazy"` except the hero and the currently viewed restaurant's cover.
- Animate only `transform/opacity`; respect `prefers-reduced-motion`.
- Keyboard: cart drawer, filters, status timeline (as a readable list, not just visual), and all forms fully operable; visible focus everywhere.
- Colour + icon + text together for every status - never colour alone.

11. Testing and QA

11.1 Server tests (new)

Scope	Cases
Auth (existing behaviour, now covered)	Register, login sets cookies, wrong password rejected, refresh rotates tokens, logout clears cookies, RBAC blocks a User from restaurantOwner-only routes
Orders + transaction fix	Placing an order clears the cart in the same transaction; simulate a failure after order creation and assert neither the order nor the cart-clear is left half-done
Uploads	Valid image type/size accepted and returns a Cloudinary URL/publicId (mock the Cloudinary SDK); oversized or wrong-type file rejected with 400
Sockets	A client that has not joined a room does not receive events for it; joining <code>order:{id}</code> for an order the user does not own is rejected
Payments (existing, now covered)	Webhook signature verification still passes/fails exactly as before the refactor (regression guard)

11.2 Client tests (new)

- Cart: adding an item from a second restaurant prompts to clear the cart first (matches backend rule).
- Checkout: price breakdown matches the mocked `GET /api/checkout` response; "Pay now" is disabled with no address selected.
- Order tracking: `OrderStatusTimeline` updates when a mocked socket event fires, and falls back to polling when the socket is torn down.
- RoleRoute: an authenticated User is redirected away from `/owner/*`; an authenticated restaurantOwner is redirected away from `/admin/*`.

11.3 Manual QA checklist

- Full order journey end to end on staging: register, browse, add to cart, checkout, pay (Cashfree sandbox), watch the status timeline update live as the owner advances it from a second browser session.
- Confirm the transaction fix: attempt to reproduce the old failure mode and confirm the order and cart stay consistent.
- Confirm cookies are `Secure` in the deployed (HTTPS) environment via browser devtools.
- Upload, replace, and remove an image on both a restaurant and a food item; confirm the old Cloudinary asset is deleted on replace.
- Keyboard-only pass through checkout and the owner order queue.

12. Build Phases and Acceptance Criteria

Phase	Tasks	Acceptance criteria
P0 Repo & hygiene	Git init, commit existing backend as-is, then apply fixes #2, #6, #7, #9 from section 8	Repo exists with a clean initial commit; server boots with correct <code>main</code> , structured logs, no razorpay dependency, and production-safe cookie flags
P1 Design system	Tokens, Tailwind config, fonts, UI primitives, three layout shells, dev-only style guide	Style guide renders every component and every StatusPill colour; layouts responsive
P2 Real-time + uploads + transactions	Fixes #1, #3, #4, #5, #8 (partial) from section 8; Socket.IO server and client wiring; seed script	Placing/cancelling an order emits a socket event a connected client receives within 2s; an image can be uploaded and appears on the food/restaurant; email worker actually sends a test email when run

Phase	Tasks	Acceptance criteria
P3 Customer app	All pages in section 5.1 wired to real endpoints and sockets	A seeded demo customer can complete a full order and watch it progress to DELIVERED
P4 Owner dashboard	All pages in section 5.2	A seeded demo owner can create a restaurant, add menu items with photos, and advance an incoming order
P5 Admin dashboard	All pages in section 5.3	A seeded demo admin can approve an owner request and see it reflected for that user
P6 Quality	Tests in section 11, accessibility and responsive QA, logging review	Test suite green; manual QA checklist complete
P7 Ship	Docker, CI, deploy, smoke test, handover note	Live URLs work end to end for all three seeded roles

13. Deployment Runbook

The database, Redis, Cashfree and Gmail SMTP already exist and are already configured locally - reuse those credentials in production rather than creating new ones, unless the owner wants separate dev/prod instances.

#	Step	Details
1	GitHub	Push the repository (this is the first-ever push - see phase P0)
2	Cloudinary	Create a free account (new for this project); copy cloud name, API key, secret
3	Render - API	New Web Service, root <code>server</code> , build <code>npm ci</code> , start <code>npm start</code> ; env vars = existing ones + the new ones in section 9.5; health check path <code>/</code> (existing) or add <code>/api/health</code> if the agent prefers a dedicated one
4	Render - worker	New Background Worker (not a web service - no HTTP port), root <code>server</code> , build <code>npm ci</code> , start <code>npm run worker</code> ; same env vars as the API
5	Seed	Run <code>npm run seed</code> once against the production database from Render Shell or locally with the production <code>MONGODB_URI</code>
6	Vercel - client	Root <code>client</code> , framework Vite, env <code>VITE_API_BASE_URL=/api</code> ; <code>vercel.json</code> rewrites <code>/api/*</code> and the Socket.IO path (<code>/socket.io/*</code>) to the Render API URL
7	CLIENT_ORIGINS	Set on Render to the final Vercel URL(s) so CORS and Socket.IO both accept it
8	Smoke test	Full order journey (section 11.3) against production

```
// client/vercel.json
{
  "rewrites": [
    { "source": "/api:path*", "destination": "https://YOUR-API.onrender.com/api:path*" },
    { "source": "/socket.io:path*", "destination": "https://YOUR-API.onrender.com/socket.io:path*" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

```
# server/Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 5000
CMD ["node", "src/server.js"]
```

14. Handover Requirements

- **README.md:** setup for both apps, how to run the worker locally, environment variables, how to run the seed script, and a summary of every backend fix applied (mapped to the numbered list in section 8).
- `.env.example` for `server/` covering existing and new variables; a short one for `client/`.
- **Handover note:** live URLs, the three seeded demo logins (never the production admin password in the repo), which placeholders (restaurant names, food photos, brand name/logo) to replace before real use, and any of the section 8 fixes that were deferred and why.
- **Quality report:** test results and the manual QA checklist outcome, plus before/after notes on the transaction and cookie fixes specifically, since those were the two "real" bugs found.

Final instruction to the build agent: the backend is already a real, working system - respect it. Fix what is broken, wire up what is unwired, and build a frontend that finally makes the whole thing usable, with the same visual polish and consistency standard as a funded consumer product.